

Architecture logicielle & systèmes distribués

[Accueil](#)[Introduction](#)[Jour 1](#)[Jour 2](#)[Jour 3](#)[Jour 4](#)[Jour 5](#)[Jour 6](#)[TP1](#)[TP2](#)[TP3](#)[Exercices](#)[Corrigés](#)[Glossaire](#)[Live Coding](#)[Dossier Architecture](#)[FAQ](#)[Support HTML modulaire](#)[Master 2](#)[6 jours](#)[3 TP progressifs](#)[Java / Spring Boot](#)

Jour 2 — Patterns d'architecture et structuration d'applications

Objectifs pédagogiques

Maîtriser les grands patterns, l'inversion de dépendance, le DDD tactique, CQRS, les Value Objects et structurer un backend Spring Boot propre.

Temps estimé

≈ 6h incluant démos et quiz

Prérequis

Jour 1 acquis — SOLID, couches, couplage

Table des matières

- [Panorama des patterns](#)
- [Architecture hexagonale](#)
- [Inversion de dépendance](#)
- [DDD tactique](#)
- [Value Objects](#)
- [Événements de domaine](#)
- [CQRS](#)
- [DTO, Mapper & Repository](#)
- [MapStruct](#)
- [Tests d'architecture](#)
- [Dossier & ADR](#)
- [Quiz](#)

Panorama des patterns d'architecture

Un pattern d'architecture est une solution **connue et éprouvée** pour un problème récurrent. Choisir un pattern c'est choisir ses avantages *et* ses inconvénients — pas de silver bullet.

Pattern	Idée centrale	Quand l'utiliser	Limite principale
Layered	Couches techniques empilées	App simple, équipe débutante	Couplage implicite entre couches
Hexagonal	Domaine au centre, ports/adapters	Testabilité forte, isolation du métier	Plus de fichiers, discipline requise
Clean Architecture	Règle de dépendance vers l'intérieur	Longue durée de vie, métier très riche	Abstractions parfois excessives
Onion	Anneaux concentriques autour du domaine	Proche de Hexagonal / Clean	Peut sembler théorique au départ
CQRS	Séparer lecture et écriture	Lectures complexes, haute scalabilité	Cohérence éventuelle à gérer
MVC	Modèle / Vue / Contrôleur	UI web classiques	Insuffisant seul pour le backend

Mode enfant 🧸

Imagine que tu construis une maison. Tu peux juste empiler des briques (Layered), ou faire un plan précis où chaque pièce a un rôle (Hexagonal), ou diviser en zones avec des règles d'accès strictes (Clean). Le plan dépend de la taille de la maison et de combien de personnes vont y vivre.

Mode pro 🎓 — Comment choisir ?

La complexité architecturale doit être **proportionnelle à la complexité métier et à la durée de vie du projet**. Un CRUD simple avec 5 tables ne mérite pas une Clean Architecture. Un calcul de risque financier qui durera 10 ans le mérite. Over-engineering et under-engineering sont tous les deux coûteux.

Layered

Web → App → Domain →
Infra
Simple, direct

Hexagonal ⭐

Domain ← Ports →
Adapters
Utilisé dans ce cours

Clean

Entities → UseCases →
Interface
Rigueur maximale

CQRS

Command ≠ Query
Scalabilité lecture

Architecture hexagonale — en profondeur

Principe fondamental

Le domaine métier **ne dépend de rien** — ni de Spring, ni de JPA, ni de Kafka. Il expose des **ports** (interfaces) et des **adapters** externes réalisent les détails techniques.

Mode enfant 🐼

Ton téléphone (le domaine) ne sait pas comment il reçoit de l'électricité. Il expose juste une prise USB-C (le port). Tu peux brancher un chargeur mural, une batterie externe ou un câble voiture (les adapters). Si tu changes de chargeur, ton téléphone ne change pas.

Structure de dossiers recommandée

```
src/main/java/fr/univ/library/
├── domain/
│   ├── model/           # Entités, Value Objects, Agrégats
│   │   ├── Book.java
│   │   ├── Loan.java
│   │   └── LoanId.java
│   ├── service/         # Services domaine (Logique pure)
│   │   └── LoanDomainService.java
│   └── event/           # Événements de domaine
│       └── LoanCreatedEvent.java
├── application/
│   ├── port/
│   │   ├── in/          # Ports ENTRANTS (use cases exposés)
│   │   │   ├── CreateLoanUseCase.java
│   │   │   └── ReturnBookUseCase.java
│   │   └── out/          # Ports SORTANTS (besoins du domaine)
│   │       ├── LoanRepositoryPort.java
│   │       └── BookAvailabilityPort.java
│   └── service/          # Implémentations des use cases
│       └── LoanApplicationService.java
└── infrastructure/
    ├── persistence/     # Adapter JPA sortant
    │   ├── JpaLoanRepositoryAdapter.java
    │   └── LoanJpaEntity.java
    ├── web/             # Adapter HTTP entrant
    │   ├── LoanController.java
    │   └── dto/
    └── messaging/        # Adapter Kafka sortant
        └── KafkaLoanEventPublisher.java
```

Port entrant — le use case

```
// Port entrant : interface définie PAR Le domaine
public interface CreateLoanUseCase {
    LoanId createLoan(CreateLoanCommand command);
}

// Commande immuable transportant Les données du use case
public record CreateLoanCommand(UUID studentId, UUID bookId) {
    public CreateLoanCommand {
        Objects.requireNonNull(studentId, "studentId obligatoire");
        Objects.requireNonNull(bookId, "bookId obligatoire");
    }
}
```

Port sortant — ce dont le domaine a besoin

```
// Ce que Le domaine REQUIERT de L'infrastructure
public interface LoanRepositoryPort {
    void save(Loan loan);
    Optional<Loan> findById(LoanId id);
    List<Loan> findActiveByStudent(UUID studentId);
}

public interface BookAvailabilityPort {
    boolean isAvailable(UUID bookId);
    void decrementStock(UUID bookId);
}
```

Service d'application — implémente le use case

```
@Service
@RequiredArgsConstructor
@Transactional
public class LoanApplicationService
    implements CreateLoanUseCase, ReturnBookUseCase {

    private final LoanRepositoryPort loanRepository;
    private final BookAvailabilityPort bookAvailability;
    private final LoanEventPublisher eventPublisher;

    @Override
    public LoanId createLoan(CreateLoanCommand command) {
        // 1. Vérifier disponibilité via Le port (abstraction)
        if (!bookAvailability.isAvailable(command.bookId())) {
            throw new BookNotAvailableException(command.bookId());
        }
    }
}
```

```
// 2. Créer l'agrégat – toutes les règles métier sont là
Loan loan = Loan.create(command.studentId(), command.bookId());

// 3. Persister et publier
loanRepository.save(loan);
bookAvailability.decrementStock(command.bookId());
eventPublisher.publish(LoanCreatedEvent.of(loan));
return loan.getId();
}
}
```

Adapter JPA — implémente le port sortant

```
@Component
@RequiredArgsConstructor
public class JpaLoanRepositoryAdapter implements LoanRepositoryPort {

    private final SpringDataLoanRepository jpa;
    private final LoanEntityMapper mapper;

    @Override
    public void save(Loan loan) {
        jpa.save(mapper.toEntity(loan));
    }

    @Override
    public Optional<Loan> findById(LoanId id) {
        return jpa.findById(id.value()).map(mapper::toDomain);
    }

    @Override
    public List<Loan> findActiveByStudent(UUID studentId) {
        return jpa.findByStudentIdAndStatus(studentId, LoanStatus.ACTIVE)
            .stream().map(mapper::toDomain).toList();
    }
}
```

✓ Règle de la flèche de dépendance

Les flèches de dépendance pointent *toujours* vers le domaine. L'infrastructure *implémente* les ports du domaine. Si vous voyez `import org.springframework` ou `import jakarta.persistence` dans votre package `domain/`, c'est une violation d'architecture.

Inversion de dépendance — le "D" de SOLID appliqué

L'inversion de dépendance (DIP) est le principe qui rend l'hexagonal possible : la logique métier dépend d'**abstractions stables**, pas d'implémentations instables.

✗ Sans inversion — couplage direct

```
...  
  
@Service  
public class LoanService {  
    // Dépendance concrète JPA !  
    @Autowired  
    private JpaLoanRepository repo;  
    // Changer de BDD → modifier ce service  
}
```

✓ Avec inversion — abstraction

```
...  
  
@Service  
public class LoanApplicationService {  
    // Abstraction – JPA, Mongo ou InMemory  
    private final LoanRepositoryPort repo;  
    public LoanApplicationService(  
        LoanRepositoryPort repo) {  
        this.repo = repo;  
    }  
}
```

Mode pro 🎓 — L'impact sur les tests

Avec l'inversion de dépendance, on injecte un **repository in-memory** dans les tests — zéro base de données, zéro contexte Spring. Un test de domaine s'exécute en **millisecondes**. Un test d'intégration avec Postgres prend 2–5 secondes. Sur 500 tests : 5 secondes vs 40 minutes.

Test du service avec doubles de test

```
...  
  
class LoanApplicationServiceTest {  
  
    private final LoanRepositoryPort loanRepo = new InMemoryLoanRepository();  
    private final BookAvailabilityPort bookAvail = new FakeBookAvailabilityAdapter();  
    private final LoanEventPublisher publisher = new NoOpLoanEventPublisher();  
}
```

```
private final LoanApplicationService service =
    new LoanApplicationService(loanRepo, bookAvail, publisher);

@Test
void createLoan_should_persist_loan() {
    LoanId id = service.createLoan(
        new CreateLoanCommand(UUID.randomUUID(), UUID.randomUUID()));
    assertThat(loanRepo.findById(id)).isPresent();
}

@Test
void createLoan_should_throw_when_book_unavailable() {
    bookAvail.setStock(someBookId, 0);
    assertThatThrownBy(() ->
        service.createLoan(new CreateLoanCommand(studentId, someBookId)))
        .isInstanceOf(BookNotAvailableException.class);
}
}
```

DDD tactique — les briques du domaine

Le **Domain-Driven Design** (Eric Evans, 2003) place le domaine métier au cœur de toutes les décisions. Ce n'est pas un pattern d'architecture, mais un ensemble de **briques tactiques** pour modéliser le domaine de façon riche et expressive.

Mode enfant 🧒

Le DDD, c'est parler le même langage que les experts métier. Si le bibliothécaire dit "prêt", "retour", "prolongation", ton code doit avoir des classes `Loan`, `Return`, `Extension` — pas `DataRecord` ou `ProcessObject`.

"The heart of software is its ability to solve domain-related problems for its users." — Eric Evans

Bounded Context

Définition

Un **Bounded Context** est une frontière linguistique où chaque terme a un sens *précis et unique*. Au-delà de cette frontière, le même mot peut avoir un sens différent.

Terme	Contexte Catalogue	Contexte Prêt	Contexte Identité
Book	Titre, auteur, éditeur, ISBN, stock	BookId uniquement (référence)	N/A
User	N/A	StudentId, quotas d'emprunt	Email, rôles, mot de passe hashé
Loan	N/A	Agrégat central avec règles métier	N/A

Agrégat & Racine d'agrégat

Règles des agrégats

- On ne modifie jamais un objet interne directement — on passe par la **racine**.
- Les transactions ne franchissent **jamais** les frontières d'un agrégat.
- Les agrégats communiquent entre eux par **identité** (UUID), jamais par référence directe.

```
// Loan est la racine – il contrôle tous ses invariants
public class Loan {
    private final LoanId id;
    private final UUID studentId;
    private final UUID bookId;
    private LoanStatus status;
    private final Instant createdAt;
    private Instant dueDate;
    private Instant returnedAt;

    // Factory method – la création valide les règles
    public static Loan create(UUID studentId, UUID bookId) {
        return new Loan(
            LoanId.generate(), studentId, bookId,
            LoanStatus.ACTIVE, Instant.now(),
            Instant.now().plus(21, ChronoUnit.DAYS), // règle : 3 semaines
            null
        );
    }

    // Méthode métier – encapsule la règle de retour
    public void returnBook() {
        if (this.status == LoanStatus.RETURNED)
            throw new BusinessException("Ce prêt est déjà clôturé.");
        if (this.status == LoanStatus.CANCELLED)
            throw new BusinessException("Un prêt annulé ne peut pas être retourné.");
        this.status = LoanStatus.RETURNED;
        this.returnedAt = Instant.now();
    }
}
```

```

    }

    // Méthode métier – est-il en retard ?
    public boolean isOverdue() {
        return status == LoanStatus.ACTIVE
            && Instant.now().isAfter(dueDate);
    }

    // Prolongation (règle : pas si déjà en retard)
    public void extend(int days) {
        if (status != LoanStatus.ACTIVE)
            throw new BusinessException("Impossible de prolonger un prêt inactif.");
        if (isOverdue())
            throw new BusinessException("Impossible de prolonger un prêt en retard.");
        this.dueDate = dueDate.plus(days, ChronoUnit.DAYS);
    }

    // Getters – PAS de setters publics (encapsulation forte)
    public LoanId getId() { return id; }
    public UUID getStudentId() { return studentId; }
    public LoanStatus getStatus() { return status; }
    public boolean isActive() { return status == LoanStatus.ACTIVE; }
}

```

⚠ Agrégat anémique — l'anti-pattern le plus répandu

```

// ❌ Agrégat anémique : juste des getters/setters, zéro logique
public class Loan {
    private LoanStatus status;
    public void setStatus(LoanStatus s) { this.status = s; } // qui contrôle les règles ?
}

// La logique se retrouve éparpillée dans les services – dupliquée partout
public class LoanService {
    public void returnBook(UUID loanId) {
        Loan loan = repo.findById(loanId);
        if (loan.getStatus() == LoanStatus.RETURNED) // règle dupliquée
            throw new Exception("...");
        loan.setStatus(LoanStatus.RETURNED); // violation d'encapsulation
    }
}

```

Résultat : règles métier dupliquées dans tous les services, invariants non garantis, bugs qui se multiplient.

Service de domaine

Quand une règle métier implique plusieurs agrégats, on crée un **service de domaine** — pur, sans état, sans framework.

```
// "Un étudiant ne peut pas emprunter plus de 5 livres simultanément"
public class LoanDomainService {

    public void validateStudentQuota(UUID studentId, List<Loan> activeLoans) {
        long count = activeLoans.stream().filter(Loan::isActive).count();
        if (count >= 5) {
            throw new QuotaExceededException(
                "Quota de 5 emprunts actifs atteint pour l'étudiant " + studentId);
        }
    }
}
```

Value Objects — des types qui ont du sens métier

Un **Value Object** est un objet défini uniquement par sa *valeur*, sans identité propre. Il est **immuable** — on ne le modifie jamais, on en crée un nouveau.

Mode enfant 🧸

Un billet de 10€ est un Value Object. Deux billets de 10€ sont identiques — ce qui compte c'est la valeur, pas "quel billet précisément". Si tu veux 20€, tu prends deux billets — tu ne modifies pas le premier.

✗ Sans Value Object — types primitifs dangereux

```
// Un UUID peut être bookId ou studentId...
// Le compilateur ne peut pas détecter l'erreur !
public void save(UUID bookId, UUID studentId) { ... }

// Bug silencieux : arguments inversés
service.save(studentId, bookId); // compile, mais faux !
```

✓ Avec Value Objects — types expressifs

```

public record LoanId(UUID value) {
    public LoanId {
        Objects.requireNonNull(value);
    }
    public static LoanId generate() {
        return new LoanId(UUID.randomUUID());
    }
}

public record BookId(UUID value) { ... }
public record StudentId(UUID value) { ... }

// Impossible de confondre BookId et StudentId
public void save(BookId bookId, StudentId studentId) { ... }

```

Value Object métier — ISBN avec validation

```

...

public record Isbn(String value) {

    private static final Pattern ISBN_13 =
        Pattern.compile("^97[89]\\d{10}$");

    // Validation à la construction – impossible de créer un ISBN invalide
    public Isbn {
        Objects.requireNonNull(value, "ISBN ne peut pas être null");
        String clean = value.replaceAll("[\\s-]", "");
        if (!ISBN_13.matcher(clean).matches())
            throw new InvalidIsbnException("Format ISBN-13 invalide : " + value);
        value = clean; // normalisation
    }

    @Override public String toString() { return value; }
}

// Usage : La validation est garantie – pas besoin de la répéter ailleurs
Book book = new Book(BookId.generate(), "Clean Code",
    new Isbn("978-0-13-235088-4")); // valide ou exception

```

Mode pro 🎓 — Value Objects et JPA

```

...

// Convertisseur JPA pour LoanId – transparent pour le reste du code
@Converter(autoApply = true)
public class LoanIdConverter implements AttributeConverter<LoanId, UUID> {

```

```
@Override public UUID convertToDatabaseColumn(LoanId id) { return id.value(); }  
@Override public LoanId convertToEntityAttribute(UUID uuid) { return new LoanId(uuid); }  
}
```

Événements de domaine

Un **événement de domaine** représente quelque chose d'important qui s'est produit dans le domaine. Nommé au **passé**, immuable, il exprime le fait accompli.

Règles des événements de domaine

- Nommé au passé : `LoanCreated`, `BookReturned`, `StudentBlocked`.
- **Immuable** — une fois créé, jamais modifié.
- Contient toutes les informations pertinentes au moment de l'occurrence.
- POJO pur ou Java record — zéro dépendance framework.

```
// Interface commune – tout événement de domaine  
public interface DomainEvent {  
    UUID eventId();  
    Instant occurredAt();  
}  
  
// Événement immuable avec Java record  
public record LoanCreatedEvent(  
    UUID eventId,  
    Instant occurredAt,  
    LoanId loanId,  
    UUID studentId,  
    UUID bookId,  
    Instant dueDate  
) implements DomainEvent {  
  
    public static LoanCreatedEvent of(Loan loan) {  
        return new LoanCreatedEvent(  
            UUID.randomUUID(), Instant.now(),  
            loan.getId(), loan.getStudentId(),  
            loan.getBookId(), loan.getDueDate()  
        );  
    }  
}  
  
public record BookReturnedEvent(  
    UUID eventId,
```

```

    Instant occurredAt,
    LoanId loanId,
    UUID bookId,
    boolean wasOverdue
) implements DomainEvent {}

```

Pattern — accumuler et publier les événements

```

// L'agrégat accumule ses événements
public class Loan {
    private final List<DomainEvent> domainEvents = new ArrayList<>();

    public static Loan create(UUID studentId, UUID bookId) {
        Loan loan = new Loan(/* ... */);
        loan.domainEvents.add(LoanCreatedEvent.of(loan)); // accumulé
        return loan;
    }

    // Consommé une seule fois par le service d'application
    public List<DomainEvent> pullEvents() {
        List<DomainEvent> events = List.copyOf(domainEvents);
        domainEvents.clear();
        return events;
    }
}

// Dans le service d'application — publié APRÈS persistance
@Transactional
public LoanId createLoan(CreateLoanCommand cmd) {
    Loan loan = Loan.create(cmd.studentId(), cmd.bookId());
    loanRepository.save(loan);
    // Publié après commit pour cohérence
    loan.pullEvents().forEach(applicationEventPublisher::publishEvent);
    return loan.getId();
}

// Handler dans le même processus (monolithe)
@EventListener
public void onLoanCreated(LoanCreatedEvent event) {
    notificationService.sendConfirmation(event.studentId(), event.loanId());
}

```

⚠ Événements intra-service vs inter-services

`ApplicationEventPublisher` est synchrone et dans le même processus — parfait pour un monolithe. Pour communiquer entre microservices distincts, il faut un broker externe (Kafka, RabbitMQ) — couvert au Jour

4.

CQRS — Command Query Responsibility Segregation

CQRS sépare les opérations d'**écriture** (Commands) des opérations de **lecture** (Queries). Les besoins de lecture et d'écriture sont souvent si différents qu'ils justifient des modèles séparés.

Mode enfant 🐼

Dans un restaurant, la **commande** (ce que tu veux manger) passe par le serveur vers la cuisine. La **carte** (menu à lire) est conçue pour être consultée rapidement par tout le monde. Ce sont deux flux totalement différents — la cuisine ne gère pas la carte.

	Command	Query
But	Modifier l'état	Lire l'état
Retourne	Rien ou un ID	Des données (jamais de modification)
Idempotent ?	Non (en général)	Toujours
Modèle	Domaine riche, agrégats	Projection optimisée, vue SQL

CQRS léger — dans un seul service

```
// == CÔTÉ COMMANDE - domaine riche ==  
  
public record CreateLoanCommand(UUID studentId, UUID bookId) {}  
public record ReturnBookCommand(LoanId loanId) {}  
  
@Service  
public class LoanCommandHandler {  
  
    public LoanId handle(CreateLoanCommand cmd) {  
        Loan loan = Loan.create(cmd.studentId(), cmd.bookId());  
        loanRepo.save(loan);  
        return loan.getId();  
    }  
  
    public void handle(ReturnBookCommand cmd) {  
        Loan loan = loanRepo.findById(cmd.loanId()).orElseThrow();  
        loan.returnBook(); // Logique dans L'agrégat  
        loanRepo.save(loan);  
    }  
}
```

```

}

// == CÔTÉ QUERY – Lecture optimisée ==

// Projection de Lecture : contient des données dénormalisées
public record LoanSummaryView(
    UUID    loanId,
    String  bookTitle,  // dénormalisé depuis catalog
    String  studentName, // dénormalisé depuis identity
    String  status,
    String  dueDate,
    boolean overdue
) {}

@Service
public class LoanQueryHandler {

    @Transactional(readOnly = true)
    public Page<LoanSummaryView> handle(GetActiveLoansQuery query) {
        // Requête SQL directe sur une vue dénormalisée – pas d'agrégat
        return loanReadRepository.findActiveByStudent(
            query.studentId(),
            PageRequest.of(query.page(), query.size())
        );
    }
}

```

Mise à jour de la projection par événement

```

// La projection est maintenue à jour via Les événements de domaine
@EventListener
@Transactional
public void on(LoanCreatedEvent event) {
    // Dénormaliser : récupérer Les données depuis d'autres services
    String bookTitle  = catalogClient.getBookTitle(event.bookId());
    String studentName = identityClient.getStudentName(event.studentId());

    LoanSummaryEntity view = new LoanSummaryEntity();
    view.setLoanId(event.loanId().value());
    view.setBookTitle(bookTitle);
    view.setStudentName(studentName);
    view.setStatus("ACTIVE");
    view.setDueDate(event.dueDate());
    loanSummaryRepository.save(view);
}

```

- ☒ Les lectures sont **beaucoup plus fréquentes** que les écritures.
- ☒ Le modèle de lecture nécessite des **jointures complexes** ou des données agrégées.
- ☒ On veut scaler la **lecture indépendamment** de l'écriture (réplicas en lecture seule).
- ☒ CRUD simple — CQRS est un over-engineering si les modèles lecture/écriture sont identiques.

DTO, Mapper & Repository — les contrats aux frontières

Élément	Rôle	Où il vit	Erreur fréquente
DTO	Transporter des données aux frontières HTTP	Infrastructure/web	Exposer l'entité JPA directement
Command	Transporter une intention d'action	Application	Y mettre de la logique métier
Mapper	Transformer DTO ↔ domaine ↔ entité JPA	Infra / application	Disperser les conversions dans les services
Repository	Accéder à la persistance	Infrastructure	Y mettre des règles métier

☒ Anti-pattern — exposer l'entité JPA en réponse HTTP

```
@GetMapping("/{id}")
public Book getBook(@PathVariable UUID id) {
    return bookRepository.findById(id).orElseThrow();
    // Problèmes :
    // 1. Détails internes JPA (@Column, lazy) sérialisés
    // 2. Changer la BDD change l'API publique
    // 3. Risque de boucle infinie JSON (relations bidirectionnelles)
    // 4. Liste des champs exposés incontrôlée
}
```

☒ Bonne pratique — DTO explicites et maîtrisés

```
// DTO d'entrée : valide la forme HTTP
public record CreateBookRequest(
    @NotBlank String title,
    @NotBlank String author,
    @Positive int stock,
    @NotBlank String isbn
) {}
```

```
// DTO de sortie : contrat d'API stable
public record BookResponse(UUID id, String title, String author,
                           int stock, String isbn, Instant createdAt) {}

// DTO de liste allégé (moins de données = moins de bande passante)
public record BookSummary(UUID id, String title, String author) {}

@GetMapping("/{id}")
public BookResponse getBook(@PathVariable UUID id) {
    return bookService.findById(id); // DTO – jamais l'entité
}
```

MapStruct — mapping sans boilerplate

MapStruct génère les mappers entre objets à la **compilation**. Zéro réflexion à l'exécution, zéro boilerplate, et les erreurs de mapping sont détectées au build.

Dépendance Maven

```
<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct</artifactId>
  <version>1.6.0</version>
</dependency>

<!-- Dans annotationProcessorPaths – après Lombok si utilisé -->
<path>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct-processor</artifactId>
  <version>1.6.0</version>
</path>
```

```
@Mapper(componentModel = "spring")
public interface BookMapper {

    // Domaine → DTO de réponse (mapping automatique par nom de champ)
    BookResponse toResponse(Book book);

    // Liste de domaine → Liste DTO
    List<BookSummary> toSummaryList(List<Book> books);

    // DTO d'entrée → domaine (id ignoré – généré par le domaine)
```

```

@Mapping(target = "id", ignore = true)
Book toDomain(CreateBookRequest request);

// Domaine → Entité JPA (LoanId → UUID)
@Mapping(source = "id.value", target = "id")
LoanJpaEntity toEntity(Loan loan);

// Entité JPA → Domaine (UUID → LoanId)
@Mapping(source = "id", target = "id",
        qualifiedByName = "uuidToLoanId")
Loan toDomain(LoanJpaEntity entity);

@Named("uuidToLoanId")
default LoanId uuidToLoanId(UUID uuid) { return new LoanId(uuid); }
}

```

✓ Ce que MapStruct génère à la compilation

```

// Code généré — ne jamais modifier manuellement
@Component
public class BookMapperImpl implements BookMapper {
    @Override
    public BookResponse toResponse(Book book) {
        if (book == null) return null;
        return new BookResponse(
            book.getId().value(), book.getTitle(),
            book.getAuthor(), book.getStock(),
            book.getIsbn().value(), book.getCreatedAt()
        );
    }
    // ... autres méthodes
}

```

Si un champ change dans le domaine et n'est pas mappé, le **build échoue** avec un message précis — vous êtes alerté immédiatement.

⚠ Pièges classiques avec MapStruct

- Oublier `componentModel = "spring"` → le mapper n'est pas un bean Spring.
- Conflit avec Lombok → mettre le processor MapStruct *après* Lombok dans `annotationProcessorPaths`.
- Mapper des entités JPA avec lazy loading → déclenche des requêtes N+1 pendant la sérialisation.

Tests d'architecture avec ArchUnit

ArchUnit vérifie **automatiquement** les règles d'architecture dans vos tests. Si quelqu'un viole une règle, le build CI casse — pas besoin de revue de code pour le détecter.

```
@AnalyzeClasses(packages = "fr.univ.library")
class ArchitectureTest {

    // Le domaine n'importe aucune classe Spring
    @ArchTest
    static final ArchRule domain_no_spring =
        noClasses()
            .that().resideInAPackage("..domain..")
            .should().dependOnClassesThat()
            .resideInAnyPackage("org.springframework..");

    // Le domaine n'importe aucune classe JPA
    @ArchTest
    static final ArchRule domain_no_jpa =
        noClasses()
            .that().resideInAPackage("..domain..")
            .should().dependOnClassesThat()
            .resideInAnyPackage("jakarta.persistence..");

    // Les contrôleurs ne touchent pas aux repositories
    @ArchTest
    static final ArchRule controllers_no_repos =
        noClasses()
            .that().resideInAPackage("..web..")
            .should().dependOnClassesThat()
            .resideInAPackage("..persistence..");

    // Les ports entrants sont des interfaces
    @ArchTest
    static final ArchRule ports_in_are_interfaces =
        classes()
            .that().resideInAPackage("..port.in..")
            .should().beInterfaces();

    // Architecture en couches vérifiée globalement
    @ArchTest
    static final ArchRule layered =
        layeredArchitecture()
            .consideringAllDependencies()
            .layer("Web") .definedBy("..infrastructure.web..")
            .layer("Application") .definedBy("..application..")
            .layer("Domain") .definedBy("..domain..")
}
```

```

.layer("Infra")        .definedBy("..infrastructure..")
.whereLayer("Web")     .mayNotBeAccessedByAnyLayer()
.whereLayer("Domain")  .mayOnlyBeAccessedByLayers("Application", "Infra");
}

```

✓ Pourquoi c'est indispensable en équipe

Un développeur junior peut involontairement injecter un `JpaRepository` dans le domaine. Sans ArchUnit, cela peut passer en revue de code. Avec ArchUnit, le build CI casse immédiatement avec un message précis. C'est du **linting architectural automatisé** — les règles implicites deviennent des tests exécutables.

Dossier d'architecture & ADR du Jour 2

📌 Travail demandé

À la fin du Jour 2 : contexte du projet, pattern choisi, diagramme ASCII, 2 ADR clés, dette technique identifiée et backlog de remboursement.

```

# Dossier d'architecture – Campus Library (Jour 2)

## Contexte
Bibliothèque universitaire : ~500 utilisateurs actifs, équipe de 3 dev,
MVP dans 4 semaines, évolution vers microservices envisagée à J+6 mois.

## Pattern choisi : Architecture hexagonale légère

### Diagramme
[ HTTP REST ] → LoanController (adapter entrant)
                |
                CreateLoanUseCase (port entrant)
                |
                LoanApplicationService (service application)
                |
                Loan / LoanDomainService (DOMAINE – zéro dépendance)
                |
                LoanRepositoryPort (port sortant)
                |
                JpaLoanRepositoryAdapter (adapter sortant JPA)

---

## ADR-001 : Architecture hexagonale plutôt que Layered

```

Statut : Accepté – 2025-09-15

Contexte : Besoin de testabilité forte du domaine métier et
préparation à un éventuel découpage en services.

Décision : Hexagonal avec ports et adapters.

Alternatives rejetées :

- Layered simple : couplage JPA dans le domaine, tests lents
- Clean Architecture : trop d'abstractions pour une équipe de 3

Conséquences :

- + Tests domaine sans Spring en millisecondes
- + Domaine portable si changement de technologie
- Plus de fichiers (ports, adapters, mappers)
- Courbe d'apprentissage initiale

ADR-002 : MapStruct pour le mapping

Statut : Accepté – 2025-09-15

Contexte : 3 couches d'objets nécessitent du mapping sans boilerplate.

Décision : MapStruct avec génération à la compilation.

Alternatives rejetées :

- Mapping manuel : verbeux, erreur-prone
- ModelMapper : réflexion runtime, moins sûr à la compilation

Conséquences :

- + Erreurs de mapping détectées au build
- Syntaxe d'annotation à apprendre

Dette technique identifiée

ID	Description	Impact	Plan
DT-01	Pas de CQRS – lecture/écriture mixées	Moyen	Sprint 3 si perf insuffisante
DT-02	Pas d'événements inter-services	Faible	Jour 4 (Kafka)
DT-03	Pas de ArchUnit en CI	Moyen	Sprint 2

Quiz de fin

1. Pourquoi l'hexagonal peut-il être plus coûteux au début mais rentable sur la durée ?

► [Voir la réponse](#)

2. Quelle est la différence entre un port entrant et un port sortant ?

► [Voir la réponse](#)

3. Que signifie "Bounded Context" et comment l'identifier ?

► [Voir la réponse](#)

4. Pourquoi un repository ne doit-il jamais contenir de règles métier ?

► [Voir la réponse](#)

5. Pourquoi utiliser des Value Objects plutôt que des String ou des UUID bruts ?

► [Voir la réponse](#)

6. Quelle est la différence entre une Command et une Query dans CQRS ?

► [Voir la réponse](#)

7. À quoi sert ArchUnit et pourquoi est-il préférable aux règles de revue de code manuelles ?

► [Voir la réponse](#)

← Jour 1

Jour 3 →